Compute

Transforming Data in AWS

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

DataCumulus Sundog Education

EC2 in Big Data

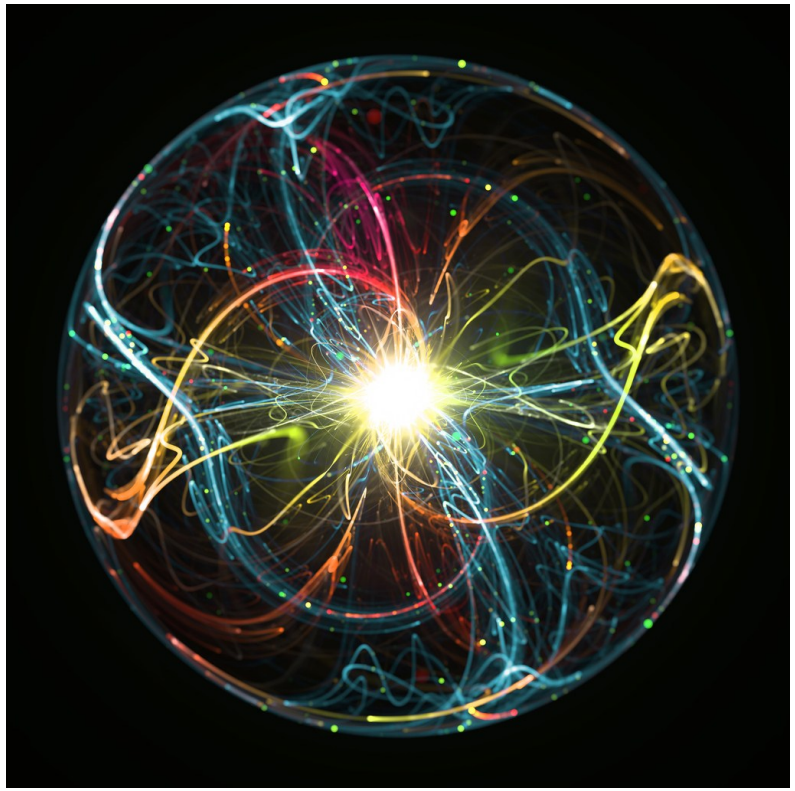
- On demand, Spot & Reserved instances:
  - Spot: can tolerate loss, low cost => checkpointing feature (ML, etc)
  - Reserved: long running clusters, databases (over a year)
  - On demand: remaining workloads
- Auto Scaling:
  - Leverage for EMR, etc
  - Automated for DynamoDB, Auto Scaling Groups, etc…
- EC2 is behind EMR
  - Master Nodes
  - Compute Nodes (contain data) + Tasks Nodes (do not contain data)

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS Graviton

- Amazon's own family of processors, powers several EC2 instance types
  - General purpose: M7, T4
  - Compute optimized: C7, C6
  - Memory optimized: R7, X2
  - Storage optimized: Im4, Is4
  - Accelerated computing (game streaming, ML inference): G5
- Offers best price performance
- Option for many data engineering services
  - MSK, RDS, MemoryDB, ElastiCache, OpenSearch, EMR, Lambda, Fargate



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS Lambda

Serverless data processing

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

What is Lambda?

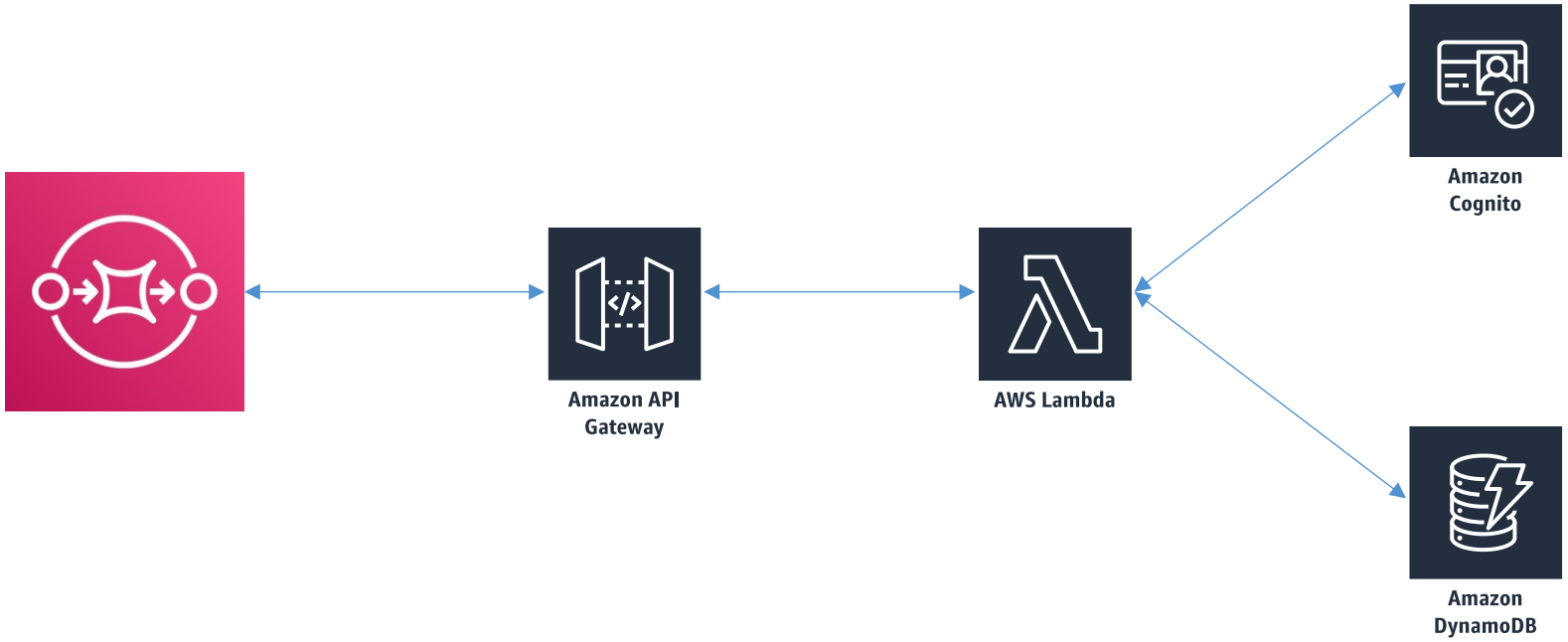
- A way to run code snippets “in the cloud”
  - Serverless
  - Continuous scaling
- Often used to process data as it’s moved around

AWS Lambda

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

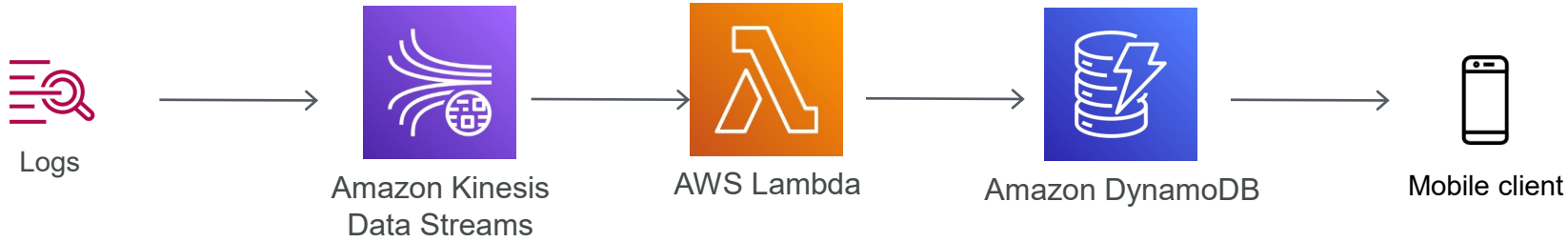
Example: Serverless Website



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Example:
Order history app



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Example:
Transaction rate alarm

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Why not just run a server?

- Server management (patches, monitoring, hardware failures, etc.)
- Servers can be cheap, but scaling gets expensive really fast
- You don’t pay for processing time you don’t use
- Easier to split up development between front-end and back-end

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Main uses of Lambda

- Real-time file processing
- Real-time stream processing
- ETL
- Cron replacement
- Process AWS events

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

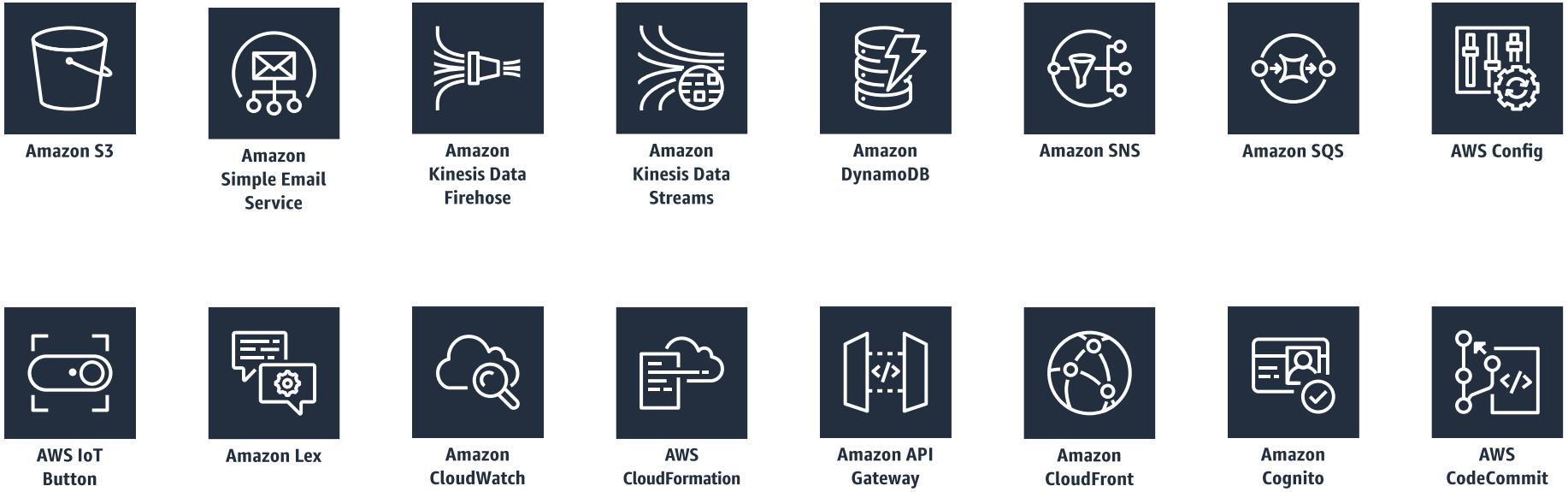
Supported languages

- Node.js
- Python
- Java
- C#
- Go
- Powershell
- Ruby

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Lambda triggers



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Lambda and Amazon Opensearch Service

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

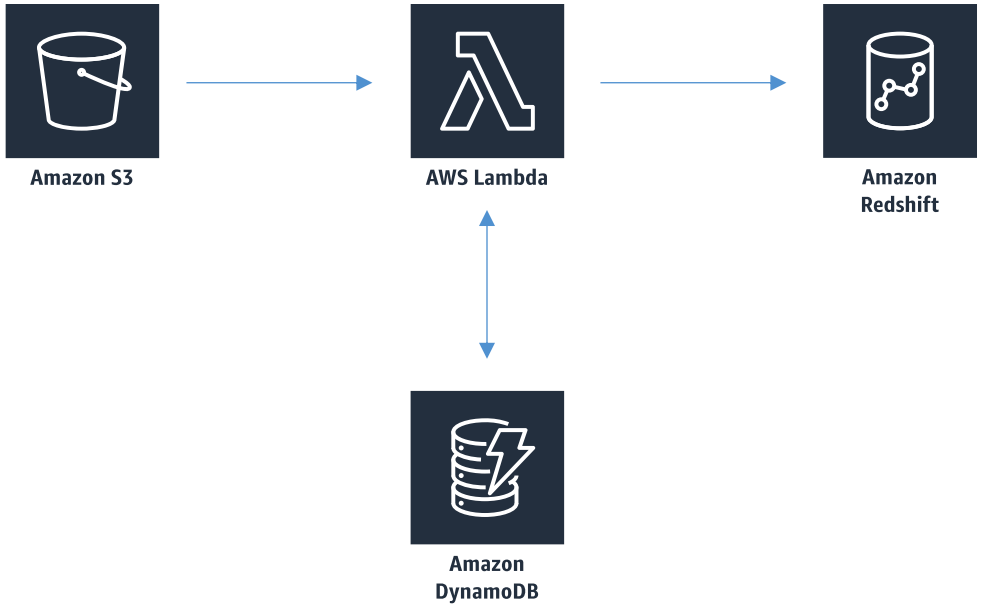
© 2023 All Rights Reserved Worldwide

Lambda and Data Pipeline

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Lambda and Redshift



Best practice for loading data into Redshift is the COPY command

But, you can use Lambda if you need to respond to new data that shows up at any time

Can use DynamoDB to keep track of what's been loaded

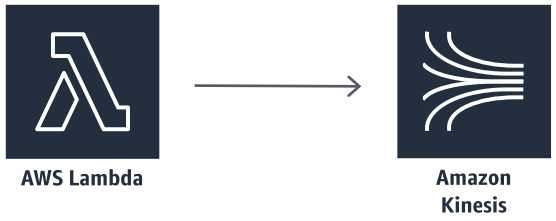
Lambda can batch up new data and load them with COPY

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Lambda + Kinesis

- Your Lambda code receives an event with a **batch** of stream records
  - You specify a batch size when setting up the trigger (up to 10,000 records)
  - Too large a batch size can cause timeouts!
  - Batches may also be split beyond Lambda's payload limit (6 MB)
- Lambda will retry the batch until it succeeds or the data expires
  - This can stall the shard if you don't handle errors properly
  - Use more shards to ensure processing isn't totally held up by errors
- Lambda processes shard data synchronously



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Cost Model

- “Pay for what you use”
- Generous free tier (1M requests / month, 400K GB-seconds compute time)
- $0.20 / million requests
- $.00001667 per GB/second

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Other promises

- High availability
  - No scheduled downtime
  - Retries failed code 3 times
- Unlimited scalability*
  - Safety throttle of 1,000 concurrent executions per region
- High performance
  - New functions callable in seconds
  - Events processed in milliseconds
  - Code is cached automatically
  - But you do specify a timeout! This can cause problems. Max is 900 seconds (15 min)

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Anti-patterns

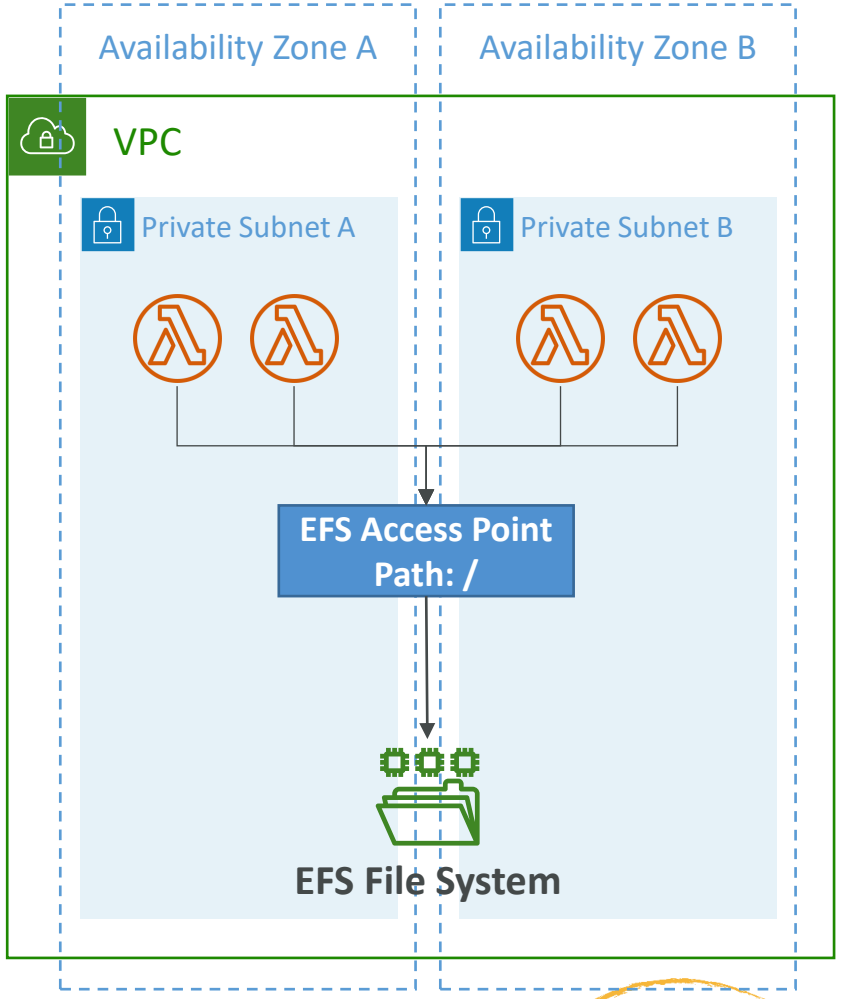
- Long-running applications
  - Use EC2 instead, or chain functions
- Dynamic websites
  - Although Lambda can be used to develop “serverless” apps that rely on client-side AJAX
- Stateful applications
  - But you can work in DynamoDB or S3 to keep track of state

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Lambda – File Systems Mounting

- Lambda functions can access EFS file systems if they are running in a VPC
- Configure Lambda to mount EFS file systems to local directory during initialization
- Must leverage EFS Access Points
- Limitations: watch out for the EFS connection limits (one function instance = one connection) and connection burst limits



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Lambda – Storage Options

	Ephemeral Storage /tmp	Lambda Layers	Amazon S3	Amazon EFS
Max. Size	10,240 MB	5 layers per function up to 250MB total	Elastic	Elastic
Persistence	Ephemeral	Durable	Durable	Durable
Content	Dynamic	Static	Dynamic	Dynamic
Storage Type	File System	Archive	Object	File System
Operations supported	any File System operation	Immutable	Atomic with Versioning	any File System operation
Pricing	Included in Lambda	Included in Lambda	Storage + Requests + Data Transfer	Storage + Data Transfer + Throughput
Sharing/Permissions	Function Only	IAM	IAM	IAM + NFS
Relative Data Access Speed from Lambda	Fastest	Fastest	Fast	Very Fast
Shared Across All Invocations	No	Yes	Yes	Yes

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS SAM

- **SAM = Serverless Application Model**
- Framework for developing and deploying serverless applications
- All the configuration is YAML code
- Generate complex CloudFormation from simple SAM YAML file
- Supports anything from CloudFormation: Outputs, Mappings, Parameters, Resources…
- SAM can use CodeDeploy to deploy Lambda functions
- SAM can help you to run Lambda, API Gateway, DynamoDB locally

DN © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS SAM – Recipe

- **Transform Header indicates it's SAM template:**
  - Transform: 'AWS::Serverless-2016-10-31'
- **Write Code**
  - AWS::Serverless::Function
  - AWS::Serverless::Api
  - AWS::Serverless::SimpleTable
- **Package & Deploy:** sam deploy **(optionally preceded by** "sam package"**)**
- Quickly sync local changes to AWS Lambda (SAM Accelerate): sam sync --watch

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Deep Dive into SAM Deployment

Build the application locally
(sam build)

Package & deploy the application
(sam deploy)

SAM Template
(YAML)

+

Application Code

transform

CloudFormation Template
(YAML)

+

Application Code

zip & upload

S3 Bucket

create/execute ChangeSet

CloudFormation

CloudFormation Stack

Lambda

API Gateway

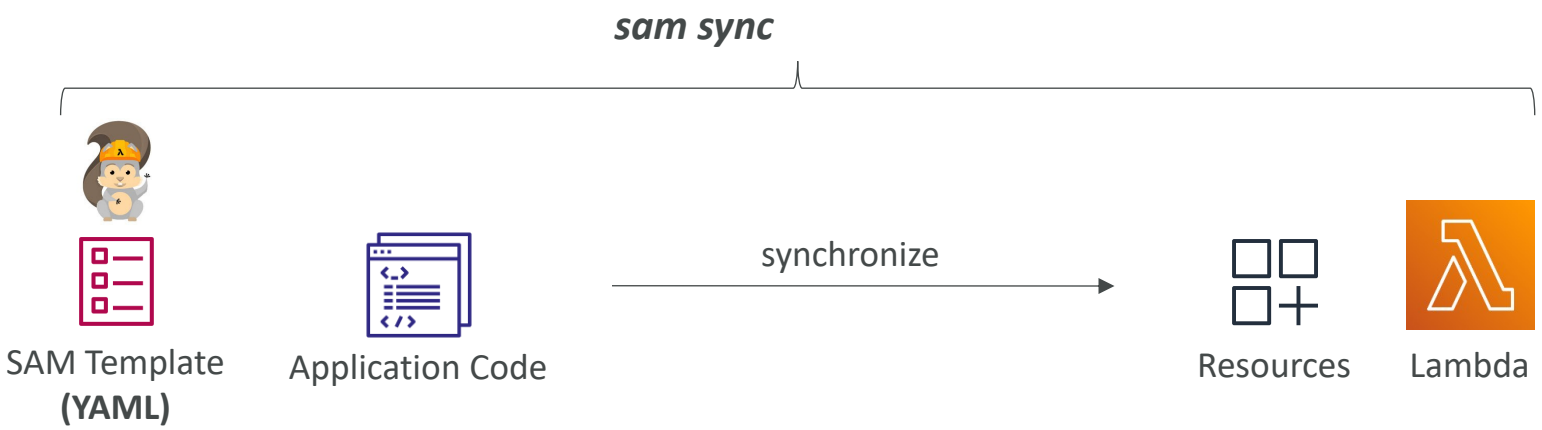
DynamoDB

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SAM Accelerate (sam sync)

- SAM Accelerate is a set of features to reduce latency while deploying resources to AWS
- ***sam sync***
  - Synchronizes your project declared in SAM templates to AWS
  - Synchronizes code changes to AWS without updating infrastructure (uses service APIs & **bypass CloudFormation**)



© 2023 All Rights Reserved Worldwide

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SAM Accelerate (sam sync) – Examples

- **sam sync (no options)**
  - Synchronize code and infrastructure
- **sam sync --code**
  - Synchronize code changes without updating infrastructure (bypass CloudFormation, update in seconds)
- **sam sync --code --resource AWS::Serverless::Function**
  - Synchronize only all Lambda functions and their dependencies
- **sam sync --code --resource-id HelloWorldLambdaFunction**
  - Synchronize only a specific resource by its ID
- **sam sync --watch**
  - Monitor for file changes and automatically synchronize when changes are detected
  - If changes include configuration, it uses **sam sync**
  - If changes are code only, it uses **sam sync --code**

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS Batch

- Run batch jobs as Docker images
- Dynamic provisioning of the instances (EC2 & Spot Instances)
- Optimal quantity and type based on volume and requirements
- No need to manage clusters, fully **serverless**
- You just pay for the underlying EC2 instances

- Schedule Batch Jobs using CloudWatch Events
- Orchestrate Batch Jobs using AWS Step Functions

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS Batch vs Glue

- Glue:
  - Glue ETL - Run Apache Spark code, Scala or Python based, focus on the ETL
  - Glue ETL - Do not worry about configuring or managing the resources
  - Data Catalog to make the data available to Athena or Redshift Spectrum

- Batch:
  - For any computing job regardless of the job (must provide Docker image)
  - Resources are created in your account, managed by Batch
  - For any non-ETL related work, Batch is probably better

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide